expect the two rightmost seats in each row to become invalid. Clicking on a seat causes the seat and the correct number of seats to its right to become held, and a similar number of seats to the left to become invalid.

I can suggest a principled argument for making the unsold or held status of a seat part of the underlying state and calculating validity on the fly. The server or whatever is storing the data eventually is only going to deal with the seats the user actually selects, whereas validity is only a concern of the UI while the user is picking seats. It's generally a good idea to separate data concerns from UI concerns.

Let's move down to the end of the component and look at the return value, which is an array `{seatItems}`. If we look a little bit further up at the line defining the `seatItems` array, we see that we are now sending four props to each seat. We sent the `key` and the `seatNumber` before; we're making sure each seat has a distinct number. Then we send a `status`, which is calculated by the `validSeatStatus` function (more about that in a moment). Our last prop is actually a function `clickHandler={onSeatChange}`—we're sending the function object `onSeatChange` itself, not calling the function.

This pattern—a parent component holding on to state for all its child components and sending functions down as props to the children to receive events that change state—is a common way in React to manage shared state. It works best if the component hierarchy isn't very deep and when the state is easily encapsulated in a set of components.

Our `Row` component has two functions for determining the status of a seat based on its number: `validSeatStatus` and `isSeatValid`. The `isSeatValid` function returns true or false based on how close the seat is to either the edge of the row or a held seat, while `validSeatStatus` converts that Boolean to a status string to send to the `Seat` component. Basically, a held seat is always `held` while an open seat might be `unsold` or `invalid` depending on its context.

We also have two functions that manage the new state of a seat after it is clicked. One of them, `newState`, was moved from the `Seat` component now that the state is